



Python Course 2025/26/2

Lecture 10: Data Visualization with Matplotlib, Seaborn, and Plotly



Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- Choose charts by question, not by habit
- Use Matplotlib with the explicit object-oriented API
- Think in mappings, marks, stats, scales, and facets
- Use Plotly when interactivity truly adds value
- Prepare data for maps and large visual workloads sensibly

Learning objective

Move from "I know how to call a plotting function" to a better engineering habit: pick the right chart, the right library layer, and the right amount of data for the reader or the browser.

By the end, you can

Explain which visual encoding matches a question, write readable Matplotlib and seaborn code, and decide when Plotly maps or aggregation-heavy workflows are worth the extra machinery.

Decision-making

Static vs interactive

Accessibility

Geospatial

Performance

Lecture 10 is about visual thinking with the modern Python data stack, not just memorizing plotting syntax.



Why Visuals Matter

Start with the question, then choose the visual encoding

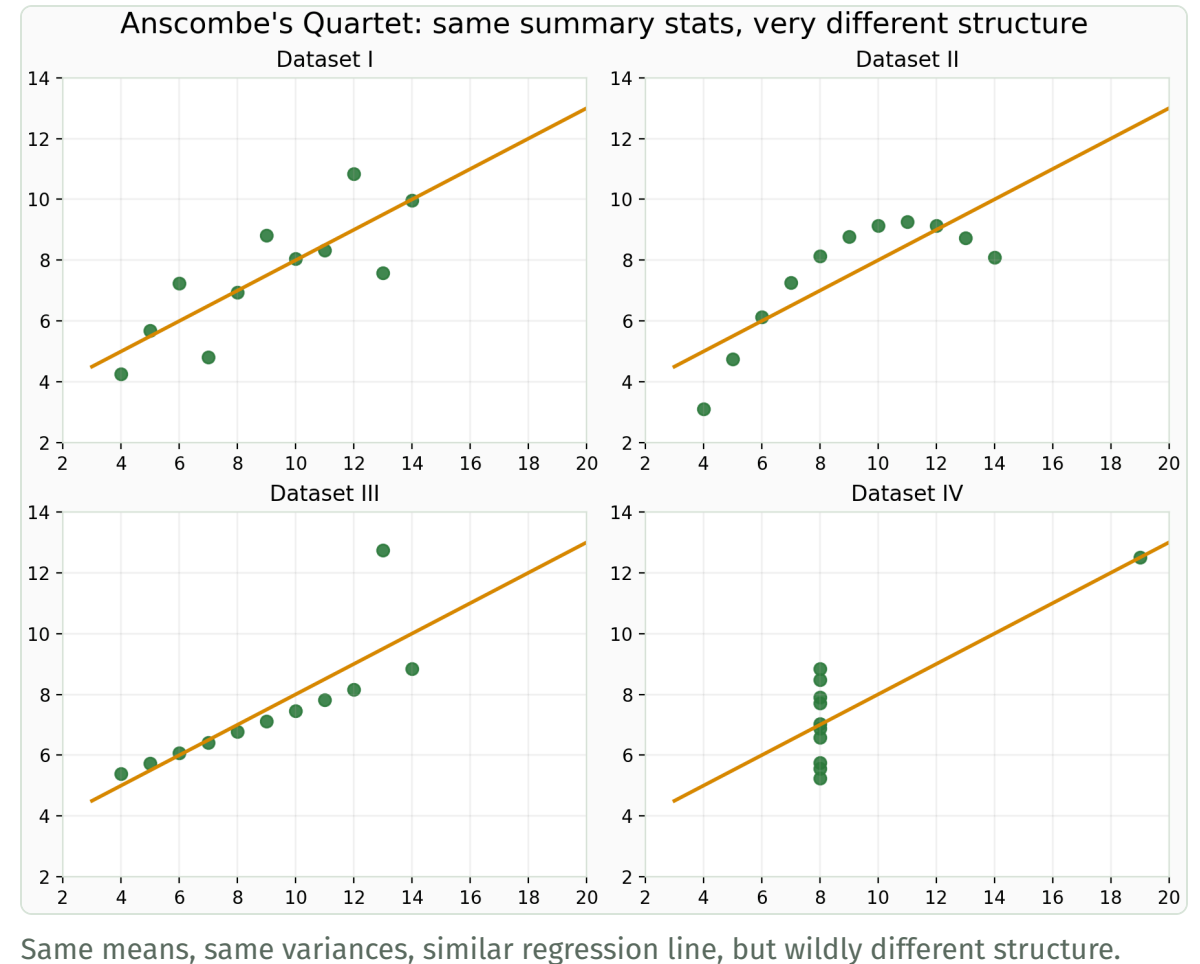
Why Visualize? Patterns Beat Raw Tables

The cognitive bandwidth problem

- Tables are read row by row and column by column: that is slow and serial.
- Plots let you see trend, spread, outliers, and clusters in parallel.
- Good charts compress many numbers into a readable shape without hiding the logic.

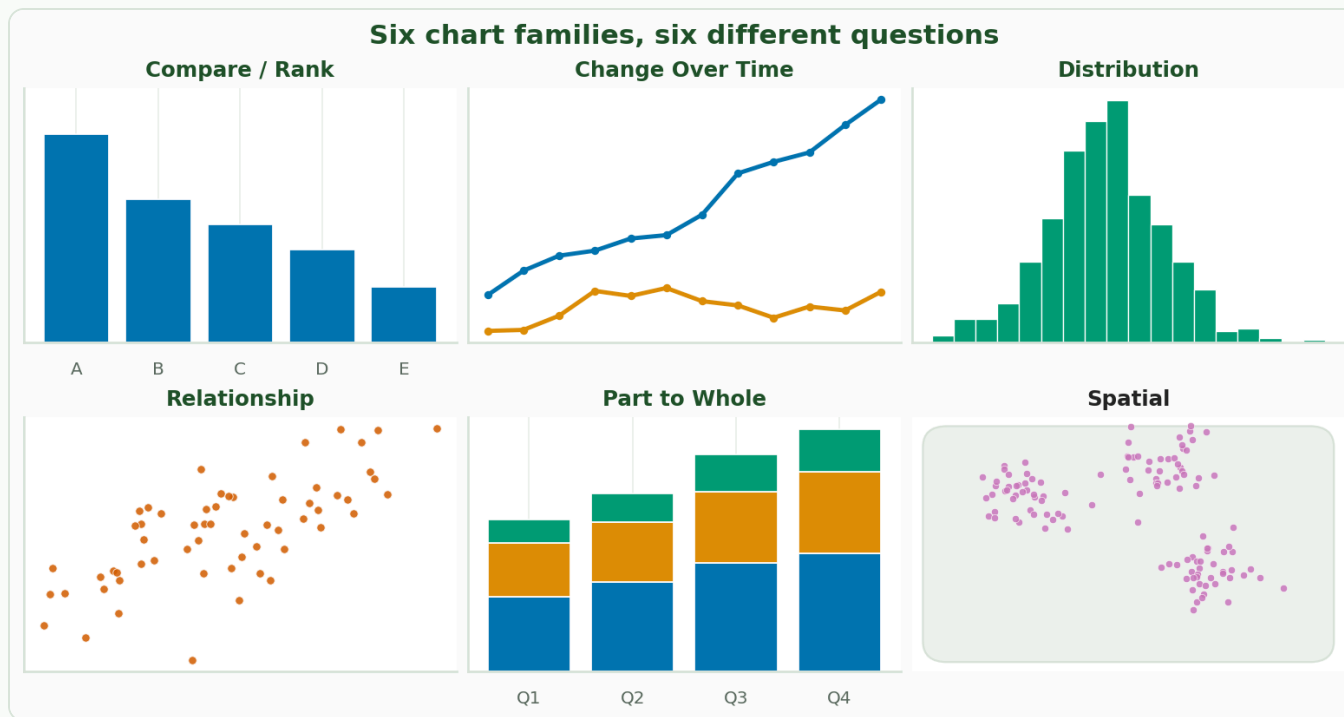
Fun fact 1

Anscombe's quartet is the classic reminder that four datasets can share almost identical summary statistics and still tell four completely different visual stories once plotted.





Choose by Question, Not by Chart Habit



Six chart families, six different questions.

Compare / Rank

Bar, dot, lollipop — categories are the story.

Change Over Time

Line or small multiples; bars only for a few periods.

Distribution

Histogram, density, box, violin, ECDF — show spread.

Relationship

Scatter, hexbin, 2D density when points overlap.

Part to Whole

Stacked bars or treemaps first; pies only for tiny n.

Spatial

Symbol map, choropleth, density map — quiet basemap.

Fun fact 2

William Playfair introduced line and bar charts in the late 18th century, and the pie chart followed in 1801. Some of our most useful chart ideas are older than computers.



Workhorse Charts Beat Fancy Charts

Default to these

- `bar` / dot / lollipop for categories
- `line` for trends over time
- `histogram` / box / violin for distributions
- `scatter` for relationships
- Small multiples when one chart gets too crowded

Use with caution or redesign

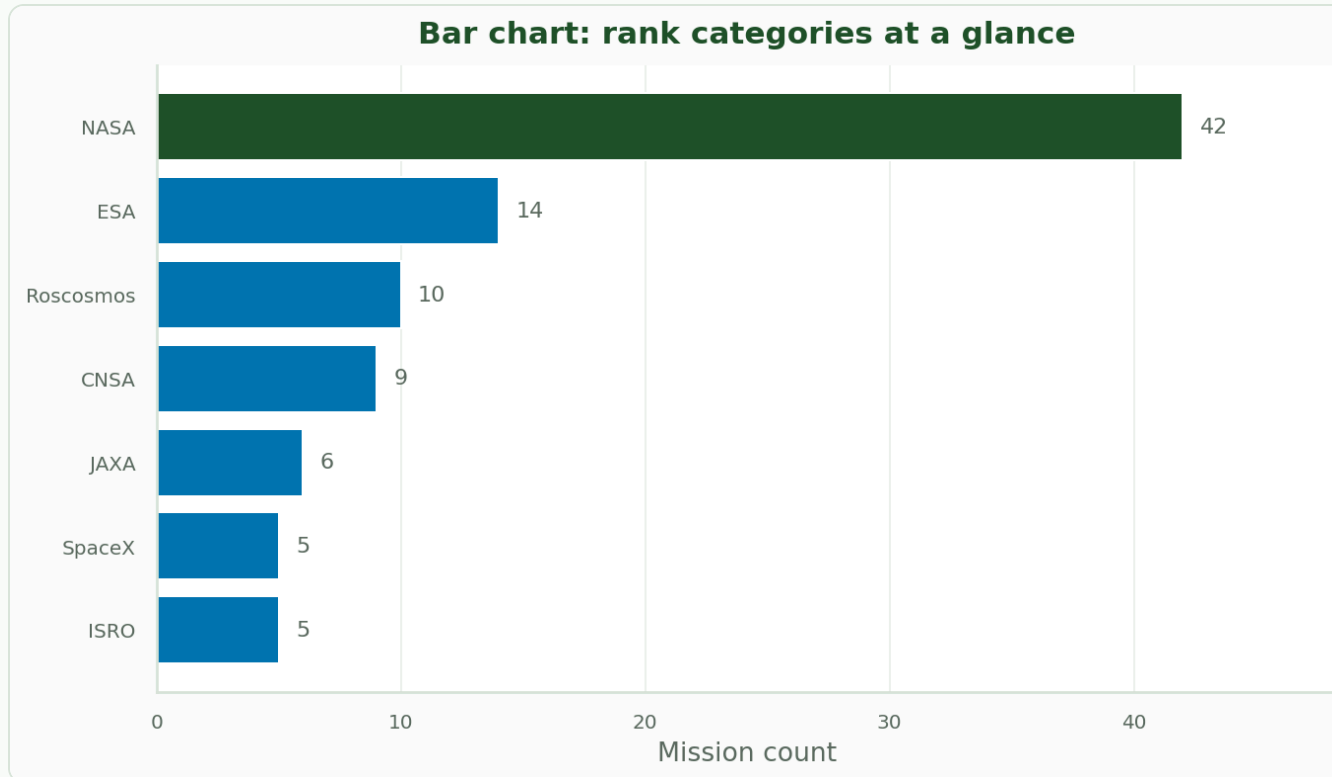
- 3D charts: decoration usually hurts reading
- Bubble charts: size is a weak perceptual channel
- Dual axes: easy to fake a relationship by scaling the axes
- Radar charts: shape changes when axis order changes
- Large pies and donuts: angles are harder to compare than lengths

Fun fact 3

Charles Minard's 1869 chart of Napoleon's Russian campaign is still widely cited as one of the greatest statistical graphics ever made because it encodes geography, direction, army size, and temperature in one coherent story.



Workhorse 1 — Bar Chart for Comparing Categories



When to reach for it

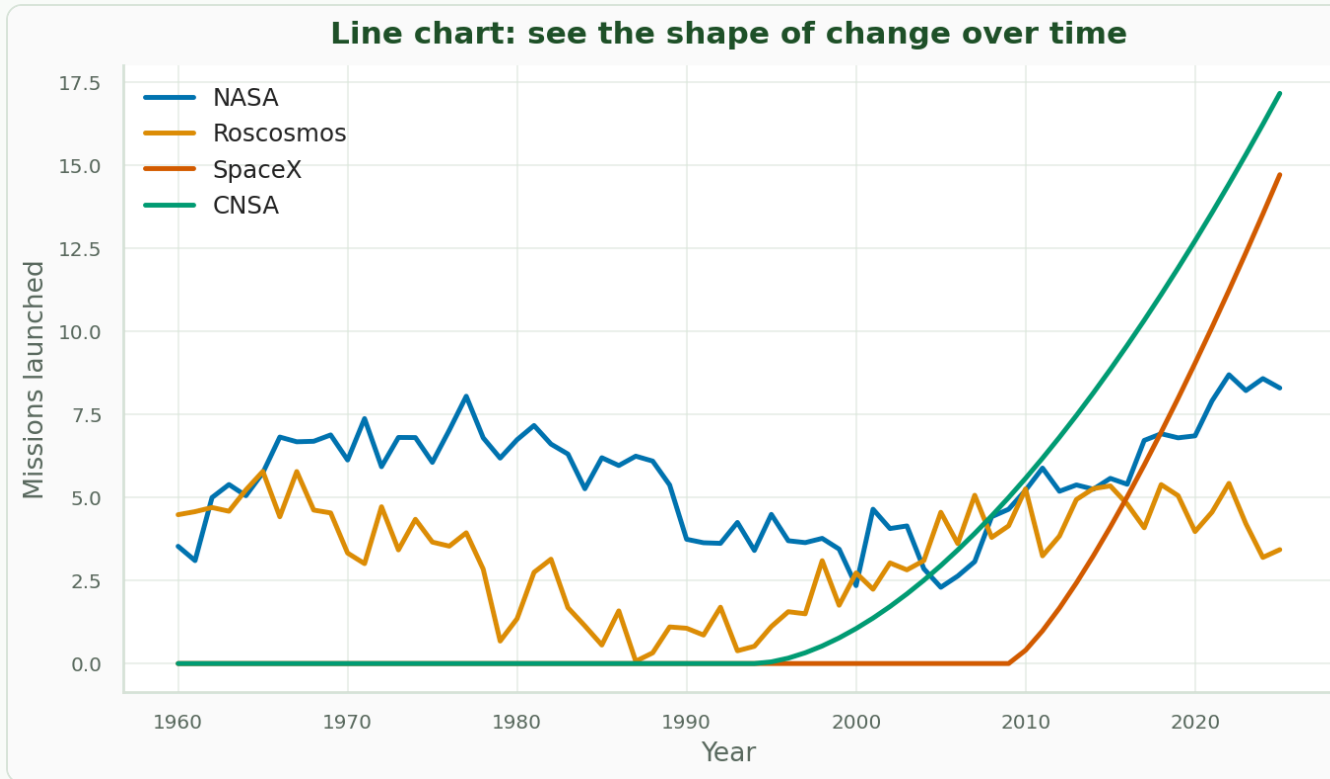
- Ranking categorical totals or rates.
- Comparing many items with long, descriptive labels.
- Putting the answer to "who is biggest?" in front of the reader.

Habits that pay off

Sort by value, not by alphabet. Start the quantitative axis at zero. Use a horizontal layout when labels would otherwise rotate.



Workhorse 2 — Line Chart for Change Over Time



When to reach for it

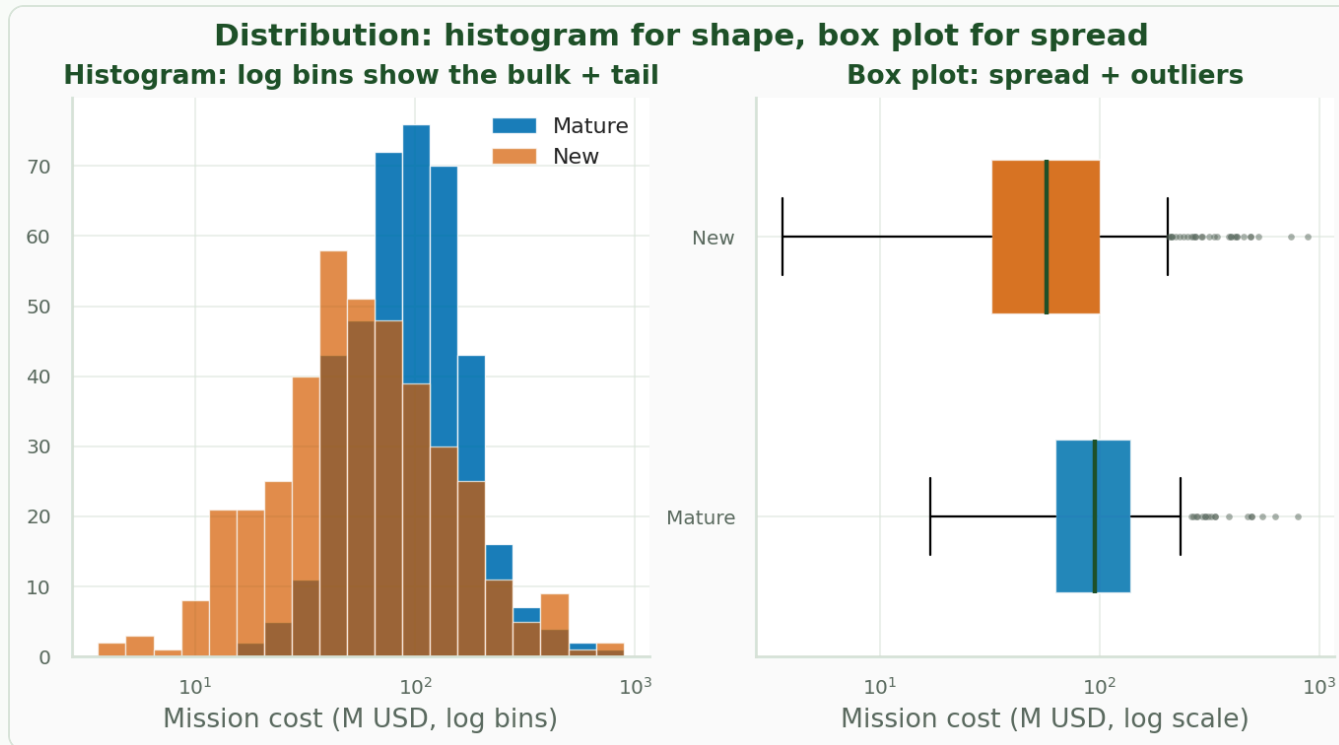
- Continuous time (years, weeks, minutes) on the x-axis.
- Multiple series with one shared scale.
- You want the eye to see direction and inflection.

Habits that pay off

Anchor the time axis to real years. Label the lines directly when there are only a few. Avoid dual axes — if scales differ, split into facets instead.



Workhorse 3 – Histogram and Box for Distributions



When to reach for it

- Asking "how is one variable spread out?"
- Comparing the shape and tail of two or three groups.
- Surfacing skew, multimodality, and outliers explicitly.

How to read the box plot

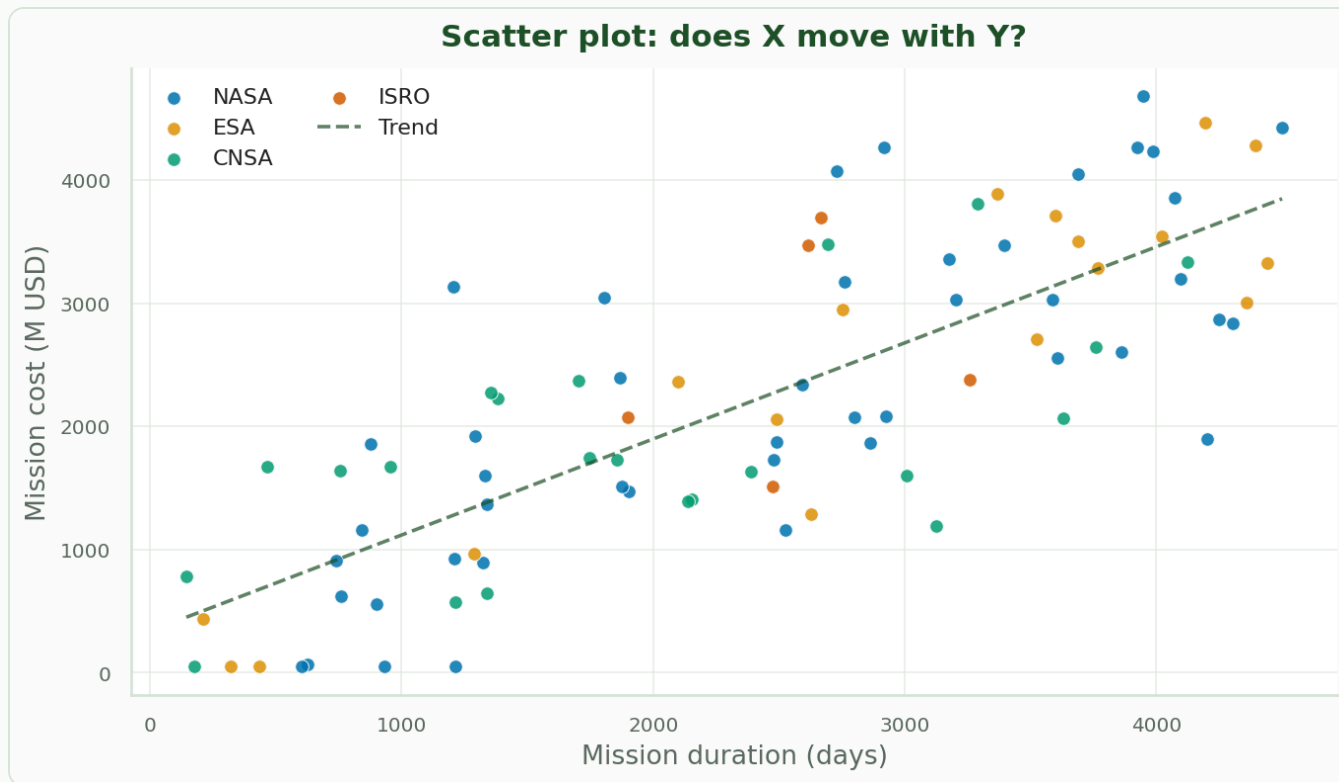
- **Median:** the thick line inside the box is the 50th percentile.
- **IQR box:** spans the middle 50% from the 25th to the 75th percentile.
- **Whiskers:** extend to the most extreme points still considered "inside the bulk" (Matplotlib defaults to 1.5×IQR past each quartile).
- **Outliers:** individual points beyond the whiskers are plotted explicitly so the tail is not hidden.
- **Log axis note:** here the x-axis is log-scaled, so distances on the page are multiplicative in dollars, not additive.

Habits that pay off

Histogram for shape, box (or violin) for spread + outliers.
Pair them when you want both stories on one slide.



Workhorse 4 — Scatter Plot for Relationships



When to reach for it

- Two continuous variables and the question "do they move together?"
- Color or shape add a third categorical variable cheaply.
- Trend lines or smoothers explain the pattern, not just the dots.

Habits that pay off

Use alpha or aggregation when points overpaint. Keep the trend line subtle. Annotate the few points that actually matter to the story.



Five-Second Sanity Checks

Ask before you export

- Can a reader answer the intended question in under five seconds?
- Does the title say the point, not only the topic?
- Are the most important values labeled or visually obvious?
- Is any decorative choice competing with the data?

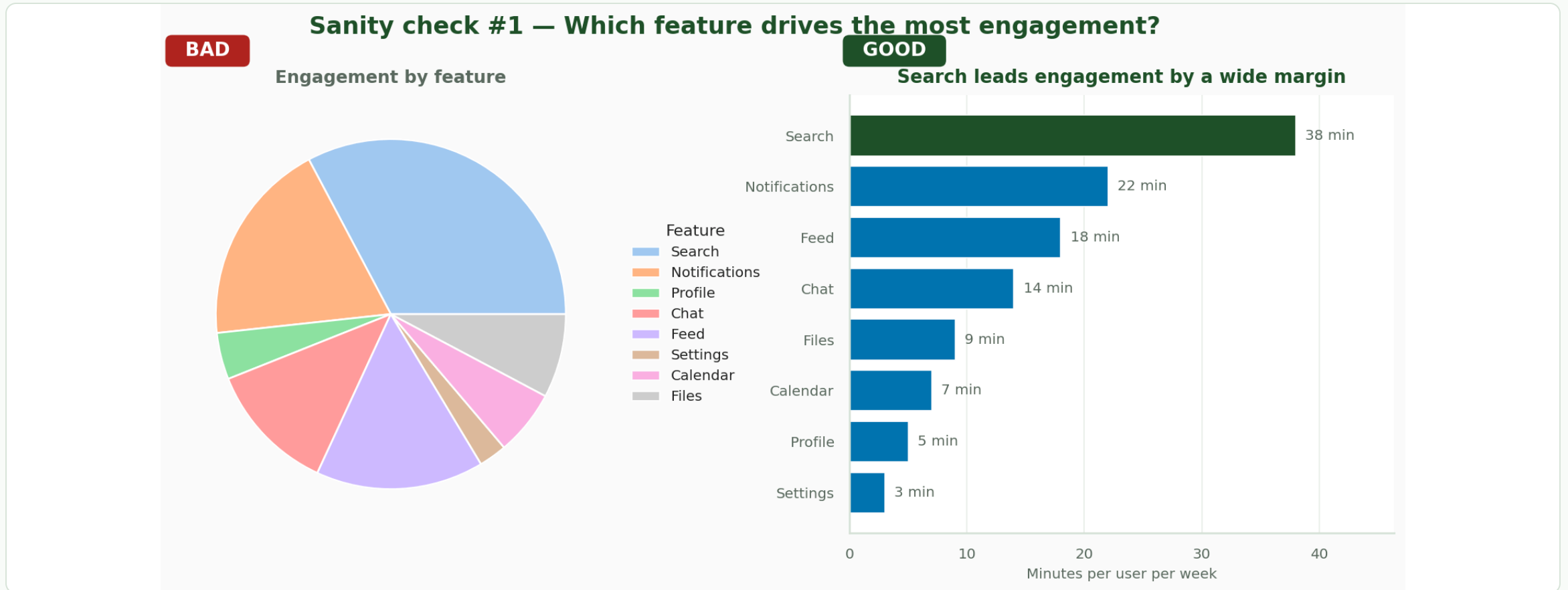
Accessibility defaults

- Use restrained palettes and direct labels when possible.
- Do not encode meaning only by color.
- For bars, start the quantitative axis at zero.
- Use low-ink gridlines and readable text, not visual noise.
- `sns.color_palette("colorblind")` is a good safe default for categories.

Safe charts feel boring only until you compare them with a misleading one. The job is not to impress the chart gallery; the job is to make the answer obvious.



Q1 — Can a reader answer in under five seconds?

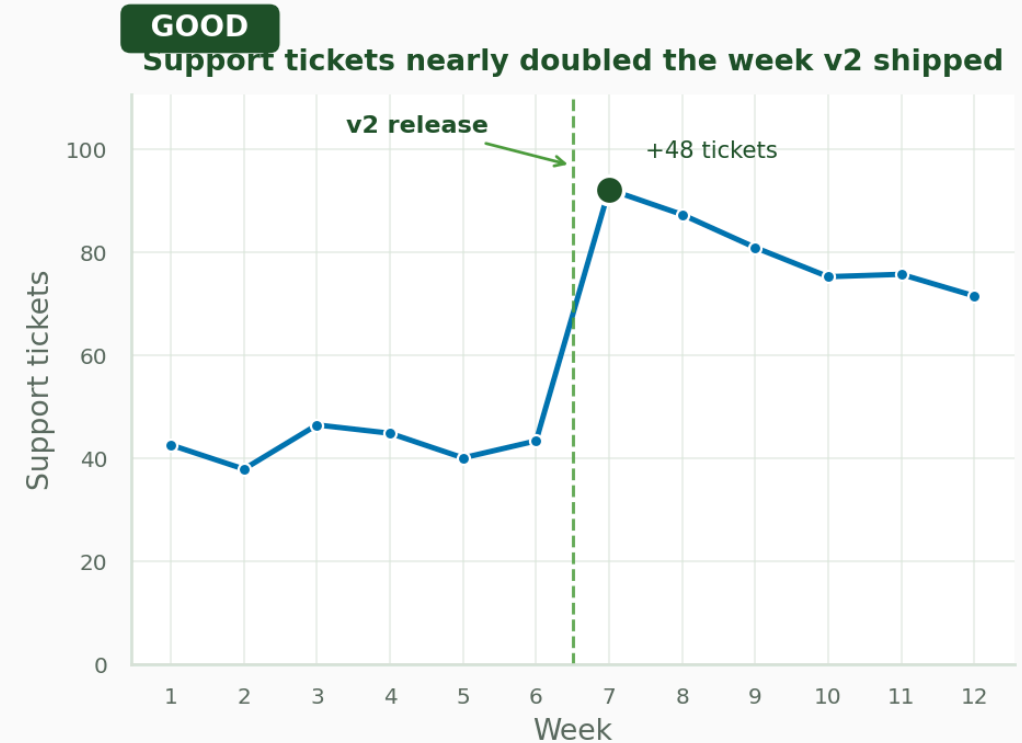
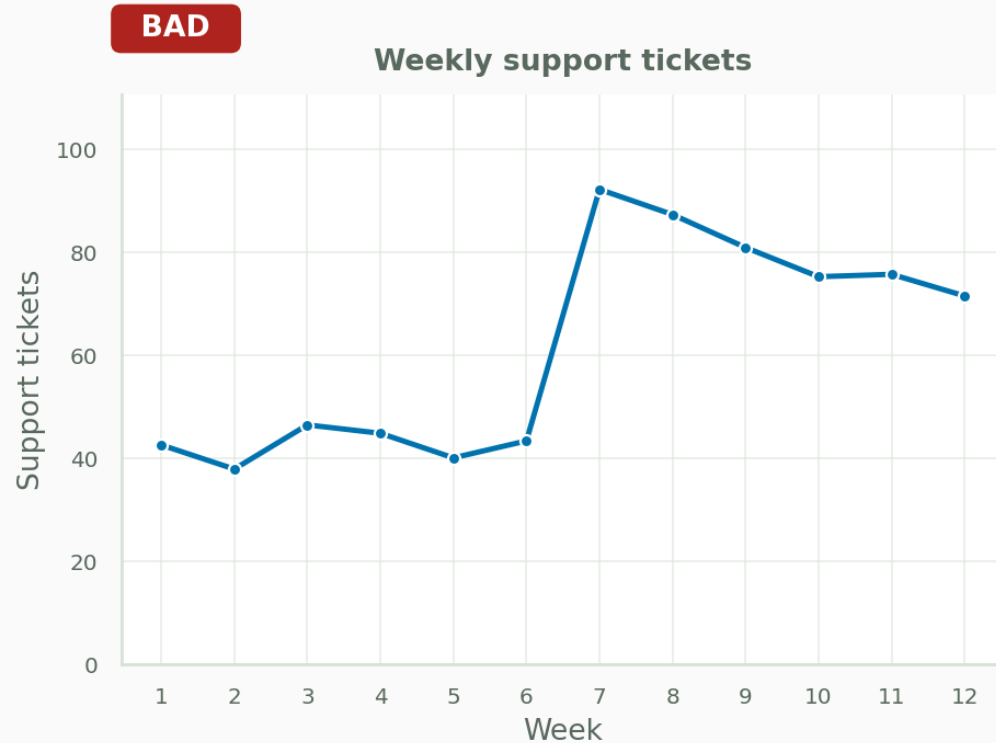


Takeaway. A pie with many similar slices and a side legend forces the reader to match colors before ranking. A sorted horizontal bar with the winner highlighted and direct labels answers "which feature wins?" in a single glance.

Q2 — Does the title say the point, not only the topic?



Sanity check #2 — Same chart, two different titles



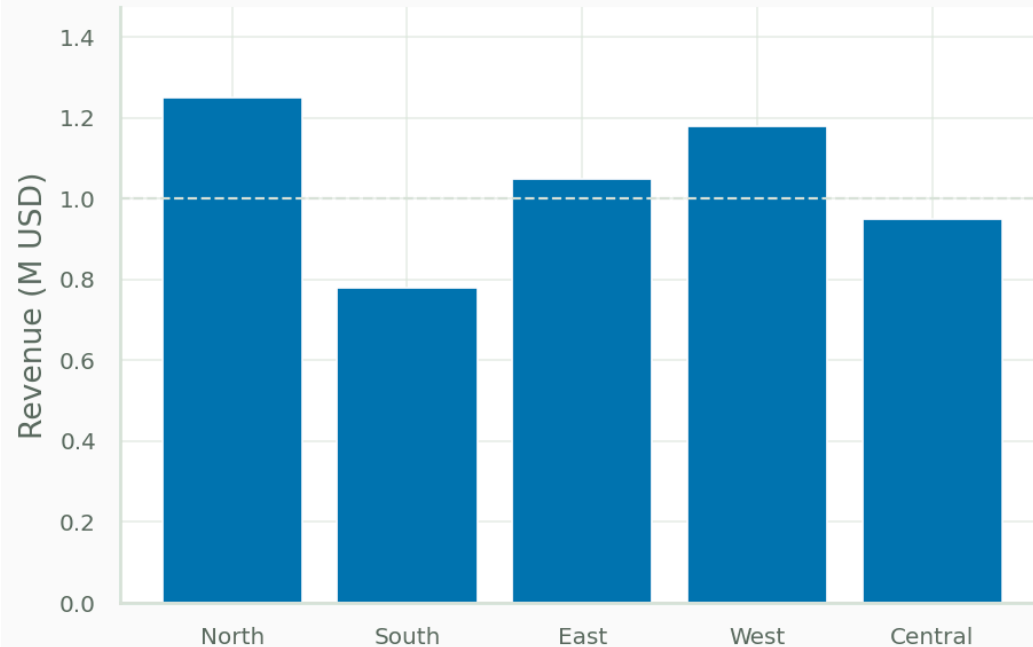
Takeaway. Identical data; only the title changes. A topic title (Weekly support tickets) makes the reader discover the story. A point title plus a light annotation hands the story over for free.

Q3 — Are the most important values labeled or visually obvious?

Sanity check #3 — Q4 revenue by region vs 1.0M target

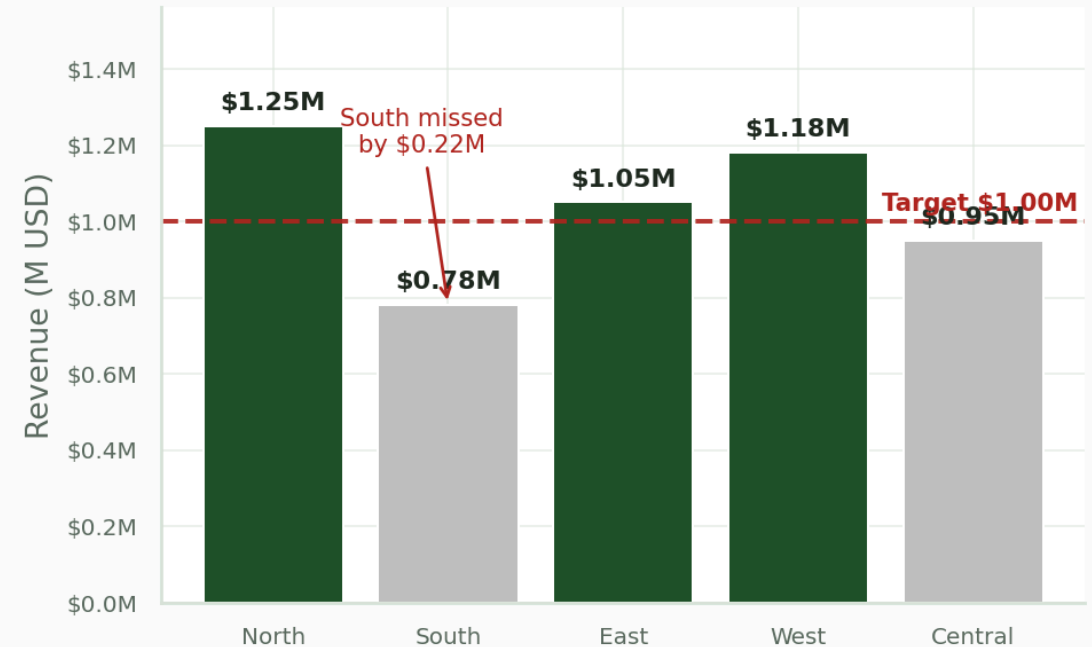
BAD

Q4 revenue by region



GOOD

Only South missed target; North beat it by 25%



Takeaway. If the key number is not labeled, colored, or annotated, it is effectively hidden. Direct labels plus one accent color for the comparison of interest make the critical value impossible to miss.

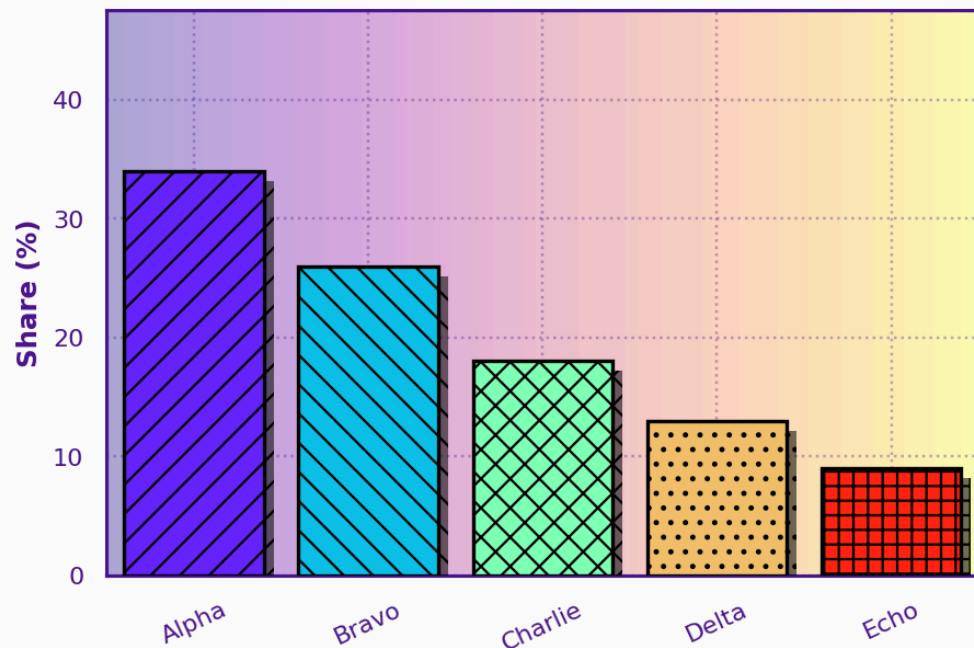
Q4 — Is any decorative choice competing with the data?



Sanity check #4 — Same numbers, different amount of ornament

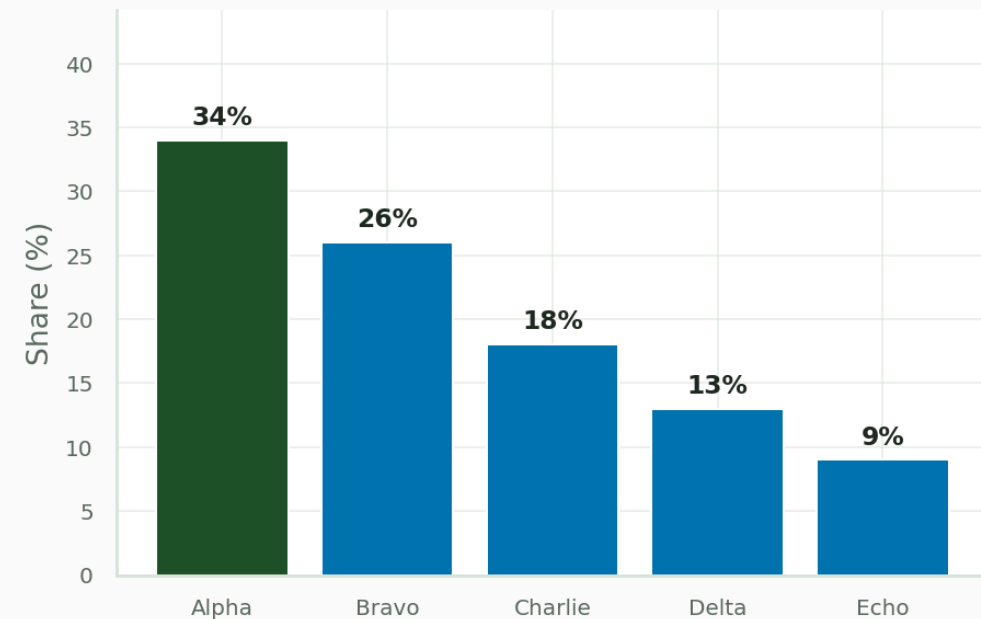
BAD

★ *Market Share by Product* ★



GOOD

Alpha leads with 34% share



Takeaway. Rainbow palettes, hatch patterns, gradient backgrounds, and heavy shadows all fight the numbers for attention. One accent color on the value that matters carries the same data with a fraction of the ink.



Checkpoint: Chart Choice

Q1

You want to show how one store's weekly revenue changes across 52 weeks. Which chart is the safest default?

- A) A line chart
- B) A pie chart with 52 slices
- C) A radar chart
- D) A contour map, for dramatic effect

Show Answer

Q2

Which chart is the best default for comparing 12 categories with long labels?

- A) Horizontal bar or dot chart
- B) Exploded 3D pie chart
- C) Dual-axis line chart
- D) Whichever chart won an argument on social media

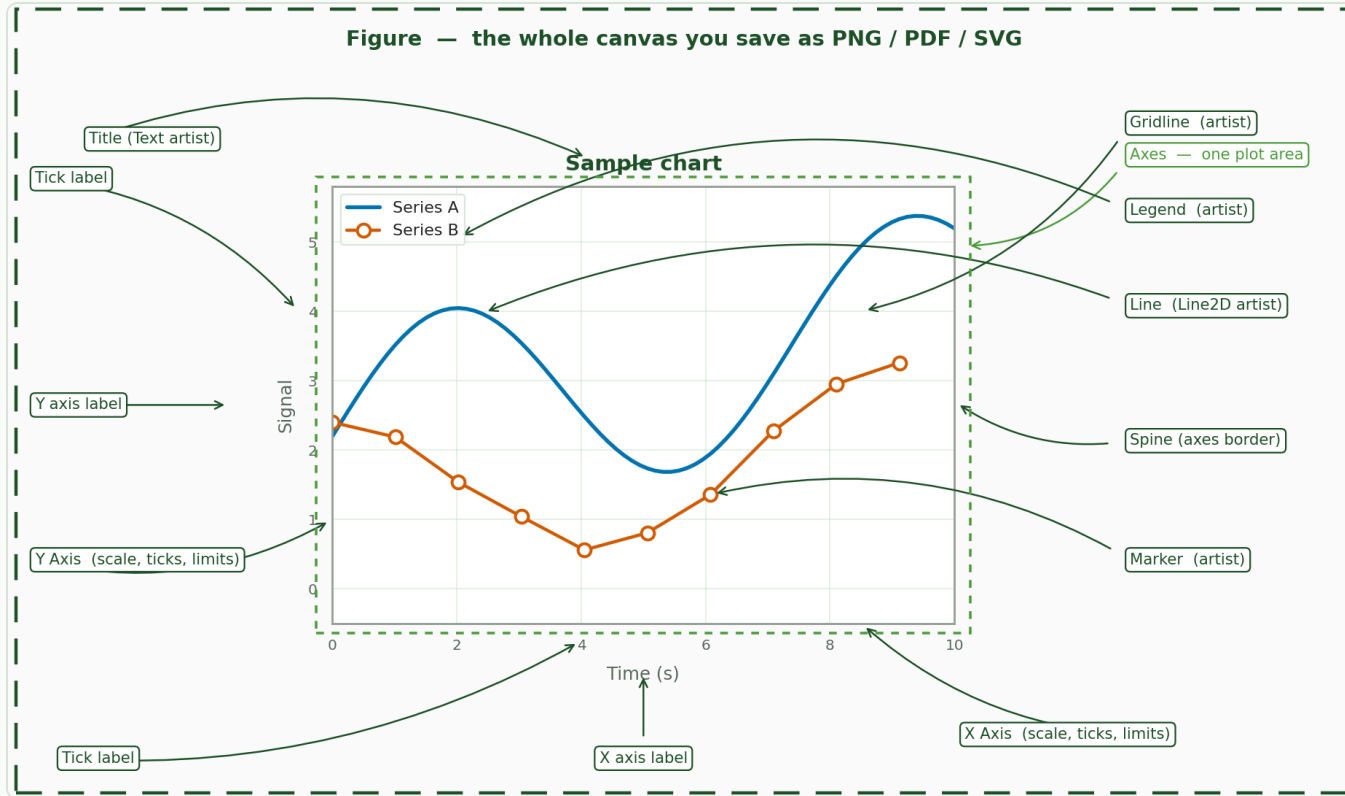
Show Answer



Matplotlib Foundations

Know the engine that higher-level libraries stand on

Figure, Axes, Axis: Know the Hierarchy



Same chart, annotated with the actual Matplotlib vocabulary.

Mental model

- **Figure**: whole canvas or exported page
- **Axes**: one plotting region
- **Axis**: scale, limits, ticks, formatting
- Everything visible is an artist attached to an axes

Even when pandas or seaborn make the chart, Matplotlib still draws the pixels.
Understanding the hierarchy is what makes debugging possible.



Prefer the Explicit Object-Oriented API

State-machine style

```
import matplotlib.pyplot as plt

plt.plot(years, revenue)
plt.title("Revenue")
plt.xlabel("Year")
plt.ylabel("USD")
```

Fine for quick sketches, but global state becomes awkward in larger figures.

OO style

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(layout="constrained")
ax.plot(years, revenue)
ax.set(title="Revenue", xlabel="Year", ylabel="USD")
```

You know exactly which axes you are modifying, which matters for subplots and reusable code.

Default lecture habit: create `fig, ax` first, then configure `ax`. It scales better than hoping Matplotlib guesses the target plot.

Semantic Layouts with `subplot_mosaic()`



Readable layout code

```
fig = plt.figure(layout="constrained")
axd = fig.subplot_mosaic([
    ["summary", "summary"],
    ["trend", "dist"],
])

axd["summary"].bar( ... )
axd["trend"].plot( ... )
axd["dist"].hist( ... )
```

Why it helps

- Named panels are easier to reason about than `axs[1, 0]`.
- Repeated names make one panel span multiple cells.
- `layout="constrained"` lets Matplotlib solve spacing for titles, legends, and labels.

Small dashboards become much less fragile when the layout is named semantically instead of managed by hard-to-remember indices.

Static Charts Are for Explanation

When Matplotlib / seaborn shine

- Papers, reports, and slides where the message is fixed
- Precise annotation and publication-quality control
- Custom layouts, small multiples, and exportable vector graphics

Export format guide

Format	Use it for
<code>.svg</code> / <code>.pdf</code>	Print, papers, infinite zoom, post-processing
<code>.png</code>	Slides, LMS uploads, screenshots
<code>.jpg</code>	Usually avoid for charts; compression adds artifacts

```
fig.savefig("mission_costs.svg", dpi=300, bbox_inches="tight")
```

SVG/PDF for publication, PNG for classroom slides, and almost never JPEG for charts.



Checkpoint: Matplotlib Foundations

Q3

Which snippet best follows the explicit Matplotlib API for a reusable chart?

- A) `plt.plot( ... ); plt.title( ... )`
- B) `fig, ax = plt.subplots( ... ); ax.plot( ... )`
- C) A handwritten screenshot of a chart
- D) Whatever worked once at 2 a.m.

Show Answer

Q4

Why is `subplot_mosaic()` attractive for complex figures?

- A) It returns named axes, which makes the layout easier to read and extend
- B) It makes legends illegal
- C) It converts plots into 3D charts automatically
- D) It only works on Tuesdays

Show Answer



Seaborn Grammar and Data Prep

Move from plotting calls to clearer plotting semantics

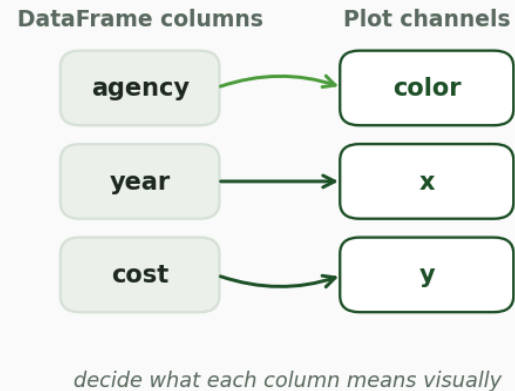
Static Stack: DataFrame -> Meaning -> Figure

1. Tidy

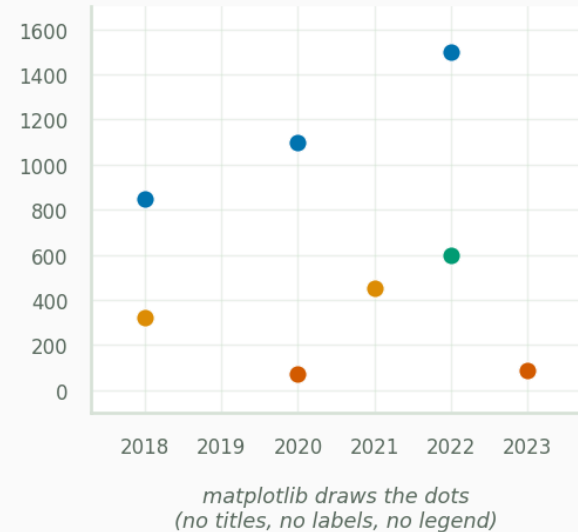
one row per mission, one variable per column

agency	year	cost (M USD)
NASA	2018	850
ESA	2018	320
NASA	2020	1100
SpaceX	2020	70
ESA	2021	450
NASA	2022	1500
CNSA	2022	600
SpaceX	2023	90

2. Map

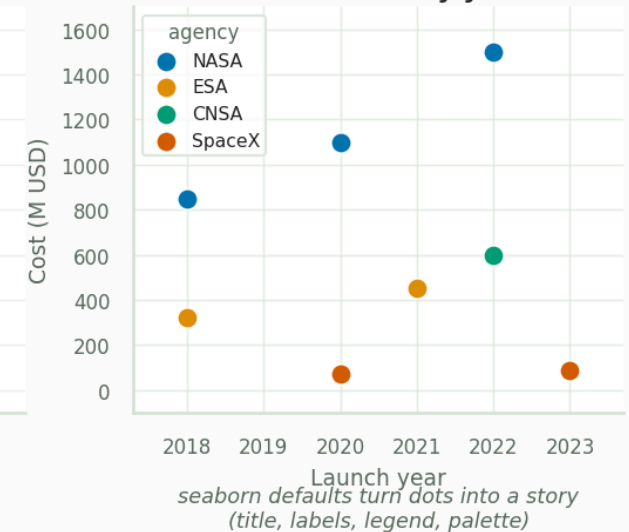


3. Render



4. Explain

Mission cost by year



Same eight rows, four stages: **tidy** shapes the data, **map** assigns meaning, **render** draws the dots, and **explain** adds the labels and legend that turn dots into a story. Use seaborn when that final step is the goal.



Where `seaborn.objects` Fits in 2026

Why it is worth learning

- Consistent property names such as `color` instead of a separate `hue` mental branch
- Composable grammar: mark + stat + move + scale
- Great for teaching plot construction explicitly

Reality check

- The classic `sns.*` API is still common and not going away.
- The official seaborn docs still describe `objects` as experimental and incomplete.
- So learn the grammar, but do not pretend the rest of seaborn disappeared.

```
import seaborn.objects as so

(so.Plot(missions, x="launch_year", y="cost_million_usd", color="agency_short")
 .add(so.Dot())
 .label(title="Mission cost by year"))
```

Demo 02 builds the same chart step by step, layering Plot → Mark → Stat → Move → Scale → Facet across ten short cells.



Marks, Stats, and Moves Are Separate Choices

Mark

Dot, Line, Bar, Bars.

What geometric shape will represent the data?

Stat

Agg, Hist, Est, PolyFit.

What transformation or summary happens before drawing?

Move

Dodge, Jitter, Stack.

How do overlapping marks separate or combine visually?

```
(so.Plot(missions, x="agency_short", y="cost_million_usd", color="decade")  
  .add(so.Bar(), so.Agg("mean"), so.Dodge()))
```

If you want a summary, ask for it explicitly. Do not rely on vague "the chart will figure it out" behavior.



Scales and Facets Rescue Clutter

When a single view fails

- Use log scale when values differ by orders of magnitude.
- Use facets or small multiples when many categories overlap.
- Use direct comparisons, not dual axes, when the audience must read the scale quickly.

Compact example

```
(so.Plot(missions, x="launch_year", y="cost_million_usd", color="agency_short")  
  .add(so.Dot())  
  .scale(y="log")  
  .facet(col="mission_class"))
```

Transform the axis when magnitude hides variation. Split the view when overlap turns the plot into spaghetti.

Fix the visual design first. If one line dominates every other line, a second axis is almost never the cleanest answer.



Data-Prep Bridge: From WKT Text to Map Columns

Vectorized parsing beats row loops

```
pattern = r"POINT Z \((?P<lon>.*?) (?P<lat>.*?) (?P<depth>.*?)\)"
coords = quakes["geometry"].str.extract(pattern).astype("float64")
quakes = quakes.join(coords)
```

Why it matters

- Plotting libraries need numeric `lon` / `lat` columns.
- Regex extraction keeps the operation vectorized and notebook-friendly.
- Once parsed, the same data can feed a static scatter, a map, or an aggregated density view.

Fun fact 5

A cell like `POINT Z (-169.6766 52.3338 10)` looks like plain text, but it is already a compact 3D geometry record waiting to become map coordinates.



Checkpoint: Static Grammar and Data Prep

Q5

What does `so.Agg("sum")` communicate in a `seaborn.objects` pipeline?

- A) Aggregate before drawing, using sum as the summary
- B) Add glitter to the bars
- C) Turn the plot interactive
- D) Ask the chart to be more aggressive

Show Answer

Q6

Which approach is the best classroom default for extracting longitude and latitude from a WKT geometry column?

- A) Loop over rows in Python and split strings manually
- B) Use `Series.str.extract(...)` with named groups
- C) Rewrite the CSV by hand
- D) Ask the projector to parse it

Show Answer



Plotly, Maps, and Performance

Interactive visuals, low-noise maps, and browser-safe workloads



Plotly Is for Exploration and Details on Demand

Use Plotly when

- You want hover details, zoom, pan, or filtering in the browser.
- The audience will explore before drawing a conclusion.
- A static PNG would collapse too much context.

Typical Plotly Express pattern

```
fig = px.scatter(  
    missions,  
    x="launch_year",  
    y="cost_million_usd",  
    color="agency",  
    hover_name="mission_name",  
    hover_data=["destination", "success"],  
)
```

The Python side describes the scene. JavaScript and WebGL render the interactive result in the browser.



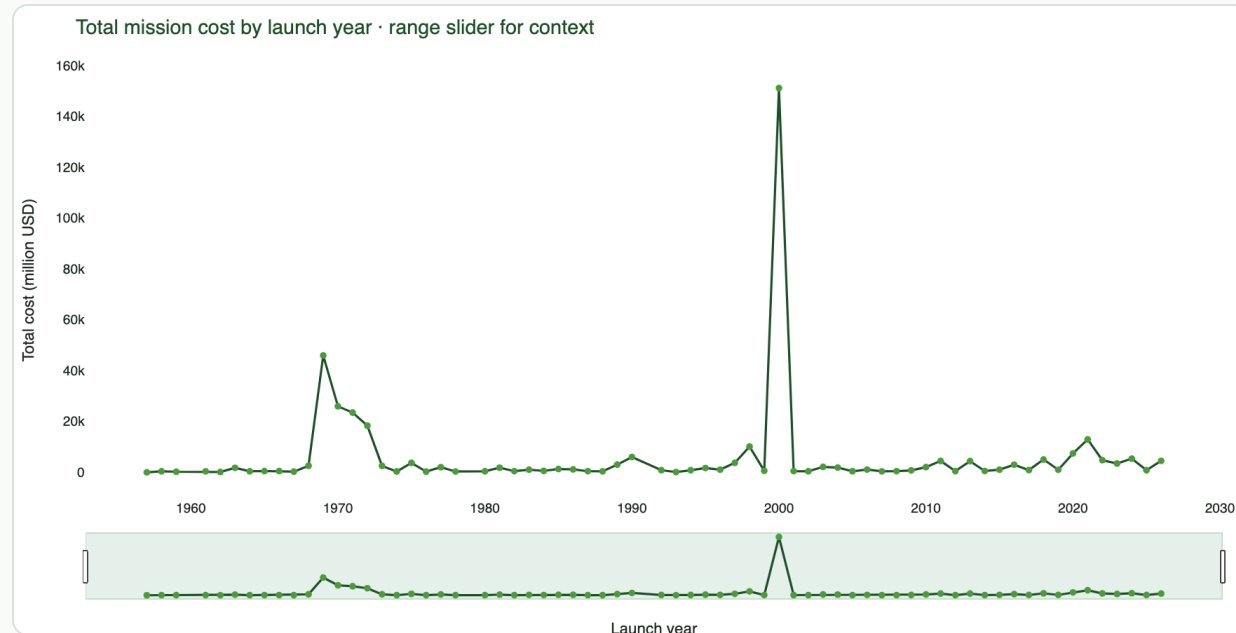
Hover, Sliders, and Focus + Context

Useful interaction patterns

- `hover_name`: one human-readable title for each point
- `hover_data`: add detail only when requested
- Range slider: show the full timeline and a zoomed window at once

Compact example

```
fig = px.line(df, x="year", y="revenue", color="sector")
fig.update_xaxes(rangeslider_visible=True)
fig.update_layout(hovermode="x unified")
```



Main view + slim navigator strip: overview first, zoom and filter, then details on demand.

Interactive features should reveal context, not replace clear design. Hover is a supplement, not a license to hide all labels and structure.

Token-Free Tile Maps with MapLibre

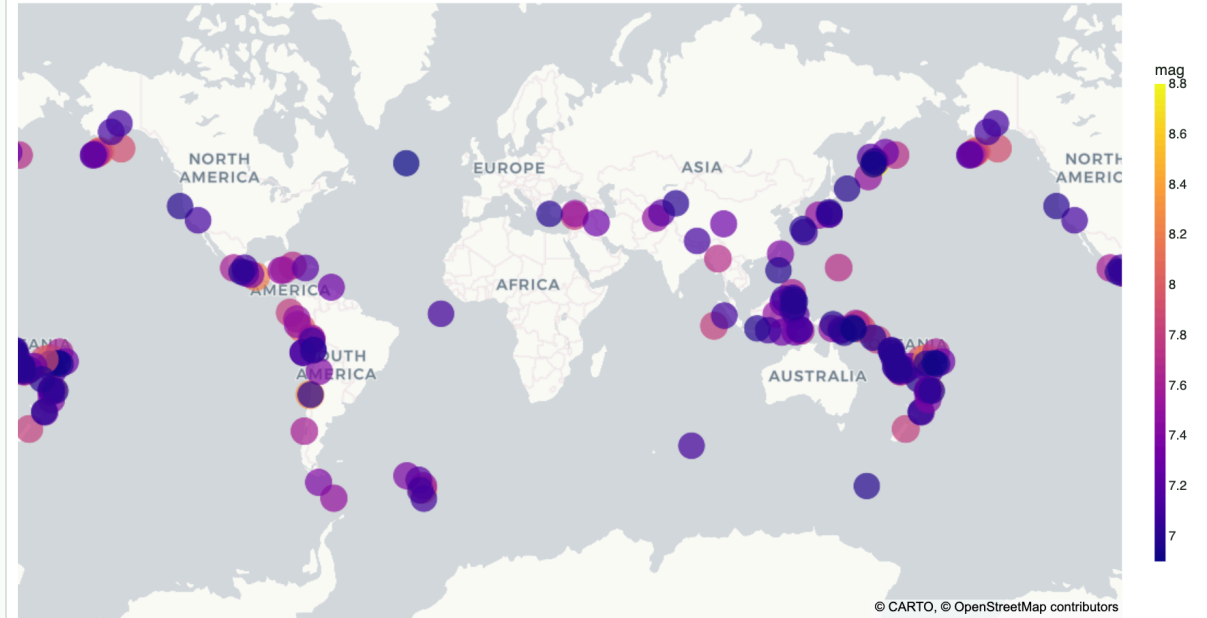
Good map defaults

- Reach for `px.scatter_map` / `px.density_map` for tile-based maps.
- Pass parsed `lat` and `lon` columns explicitly.
- Pick quiet tiles such as `carto-positron` or `carto-darkmatter`.
- Reserve color and size for meaningful data, not for map decoration.

Typical pattern

```
fig = px.scatter_map(
    quakes,
    lat="lat", lon="lon",
    size="mag", color="mag",
    hover_name="title",
    map_style="carto-positron",
    zoom=0.7,
)
```

Largest earthquakes · `px.scatter_map` + `carto-positron`



Largest earthquakes rendered with `px.scatter_map` on the `carto-positron` MapLibre basemap.

Fun fact 6

Plotly's tile maps render through MapLibre, so the built-in styles work straight out of the box — no API token, no signup, no surprise paywall mid-demo.



Large Data: Aggregate Before You Render

Browser-safe strategy

- Sample if the question tolerates approximation.
- Bin or aggregate when overlap destroys meaning.
- Use density views for huge spatial or scatter workloads.
- Let pandas / Polars do the heavy lifting upstream.

Do not send one million raw points to the browser just because you can. Make the compute layer summarize first, then let the visual layer explain.

1

Read / scan

2

Filter early

3

Aggregate

4

Hand off

5. Render the smaller result

Plotly should see an answer-shaped table, not the whole warehouse.

Special Cases: One Good Pattern, Two Risky Ones

Population pyramid: good special case

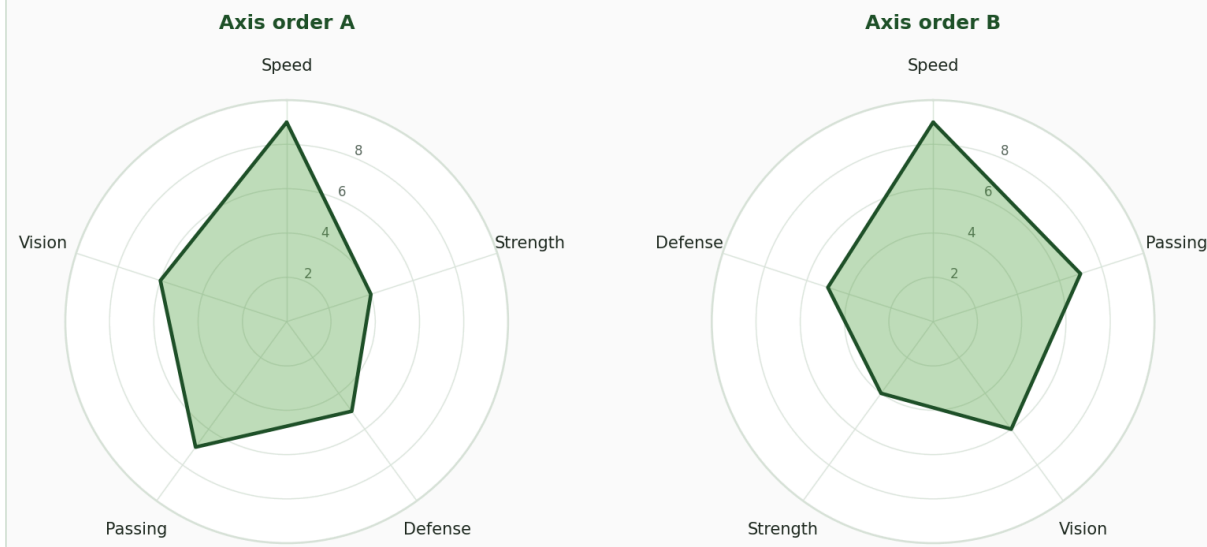
- Mirror horizontal bars around a central zero axis.
- Label the x-axis with absolute values so readers do not think negative people exist.
- Great for age structure comparisons.

Dual axes: only use them when the relationship between two different units is the story and you can color-code axes honestly. Most of the time, split the view instead.

Export formats

`fig.write_html( ... )` for interactive sharing.
`fig.write_image( ... )` for static PNG / SVG export (with Kaleido).

Same values, different radar shape after reordering axes



Fun fact 4

Radar charts can change their visual impression simply by reordering the axes. The values stay the same, but the shape feels different, which is one reason they are a risky default.



Checkpoint: Interactive Visualization

Q7

You need to show 500,000 earthquake points on a map. What is the best first reaction?

- A) Send every point to the browser and hope the laptop forgives you
- B) Consider sampling, binning, or density aggregation first
- C) Convert the map into a pie chart
- D) Pretend the browser tab crash is an advanced feature

Show Answer

Q8

Which Plotly Express map function is the current default for token-free tile maps?

- A) `px.scatter_map`
- B) `px.scatter_mapbox`
- C) `px.globe_of_doom`
- D) The one from a 2023 tutorial copied without reading the warning

Show Answer



Lecture Summary

Question First

- Pick the chart family from the question, not from habit
- Prefer workhorse charts over decorative ones
- Check readability and accessibility before export

Static Stack

- Use Matplotlib's OO API as the foundation
- Use seaborn for clearer semantics and plotting grammar
- Transform the data shape before asking the chart to explain it

Interactive Stack

- Use Plotly when hover, zoom, or maps add real value
- Aggregate before plotting large datasets
- Choose HTML for exploration, SVG/PDF/PNG for delivery

The real skill is not knowing three plotting libraries. It is deciding what the reader should notice, then sending exactly enough data through the right layer of the stack to make that answer obvious.



Thank You for Your Attention

"The greatest value of a picture is when it forces us to notice what we never expected to see." - John Tukey

Chart choice

Matplotlib

Seaborn

Plotly

Maps

Accessibility